

课教师：柯嵩宇

2025 年 5 月 27 日



福州大学
FUZHOU UNIVERSITY

① 绪论

为什么要学习编译原理？ 为什么需要编译程序？ 编译程序的作用与构成

② 语言与文法

③ 词法分析与正则文法

① 绪论

为什么要学习编译原理?

为什么需要编译程序?

编译程序的作用与构成

② 语言与文法

③ 词法分析与正则文法

[illegible]

- 自动机)、语法分析 (LL/LR 算法)、中间代码生成 (中间代码优化) 到目标代码生成, 是形式语言、算法、体系结构、性能 (Performance) 可深入理解解释执行与编译执行的性能权衡, 对性能控制有助于编写高效 JavaScript 代码。

对个人发展而言

- 理解计算系统全栈原理

- 编译原理贯穿词法分析（正则自动机）、语法分析（LL/LR 算法）、中间代码生成（SSA 形式）、优化（数据流分析）到目标代码生成，是形式语言、算法、体系结构的综合实践。
- 学习 JIT 编译（如 Java HotSpot）可深入理解解释执行与编译执行的性能权衡，掌握 V8 引擎的隐藏类优化机制有助于编写高效 JavaScript 代码。

- 打破编程语言的神圣性

- 掌握编译原理后，程序员会意识到所有编程语言本质上都是形式化符号系统。例如：
Python 的装饰器（@decorator）不过是语法糖，编译阶段会被展开为高阶函数调用。
Java 的 Lambda 表达式在字节码层面被编译为匿名内部类的 invokedynamic 指令。这种祛魅过程让开发者不再被语言特性束缚，敢于根据需求改造工具链。如 Netflix 改造 JVM 实现无暂停 GC，本质是深入理解编译与运行时机制。

[illegible]

- 文件格式与反编译技术（如 IDA Pro 的 CFG 恢复算法调用约定（如 x86 的 stdcall 与 fastcall）。
- 分析二进制序列化代码（如 Protobuf 编译器生成漏洞（如 DAO 攻击）往往源于 EVM 字节码生成

对个人发展而言

- 逆向工程与安全分析的核心钥匙

- 病毒分析：理解 PE、ELF 文件格式与反编译技术（如 IDA Pro 的 CFG 恢复算法），需掌握编译器生成的函数调用约定（如 x86 的 stdcall 与 fastcall）。
- 协议破解：网络协议逆向常需分析二进制序列化代码（如 Protobuf 编译器生成的编解码函数）。
- 区块链安全：以太坊智能合约漏洞（如 DAO 攻击）往往源于 EVM 字节码生成阶段的未定义行为。

- 培养系统级问题解决能力

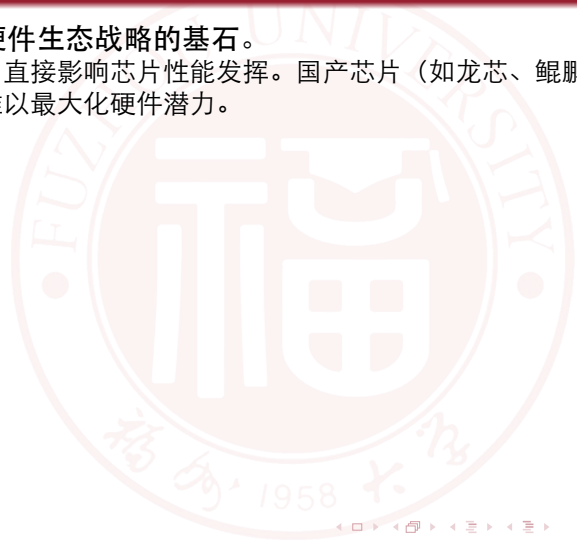
- 调试复杂问题需多层级视角。例如内存泄漏可能源于编译器逃逸分析缺陷，性能瓶颈可能与循环优化策略相关。
- 编译器开发需要兼顾类型系统设计、内存安全机制与平台 ABI 兼容性，锻炼工程化思维。

文件格式与反编译技术（如 IDA Pro 的 CFG 恢复算法调用约定（如 x86 的 stdcall 与 fastcall）。
 分析二进制序列化代码（如 Protobuf 编译器生成的漏洞（如 DAO 攻击）往往源于 EVM 字节码生成阶段。例如内存泄漏可能源于编译器逃逸分析缺陷，性能设计、内存安全机制与平台 ABI 兼容性，锻炼工程的设计（如 TensorFlow XLA），要求融合领域知识与

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡

[illegible]

- **编译程序是构建自主、可信软硬件生态战略的基石。**
 - 编译器是软件与硬件的桥梁，直接影响芯片性能发挥。国产芯片（如龙芯、鲲鹏、申威）若缺乏自主编译器，难以最大化硬件潜力。



- 硬件生态战略的基石。
- 直接影响芯片性能发挥。国产芯片（如龙芯、鲲鹏）以最大化硬件潜力。
- 研制、生产、维护，关键技术不受制于他人，能够可靠性、供货的稳定性。

件生态战略的基石。

直接影响芯片性能发挥。国产芯片（如龙芯、鲲鹏）以最大化硬件潜力。

研制、生产、维护，关键技术不受制于他人，能够可靠性、供货的稳定性。

操作系统、编译器、数据库、网络（软硬件）。

-

件生态战略的基石。

直接影响芯片性能发挥。国产芯片（如龙芯、鲲鹏）以最大化硬件潜力。

研制、生产、维护，关键技术不受制于他人，能够可靠性、供货的稳定性。

操作系统、编译器、数据库、网络（软硬件）。

- 件生态战略的基石。
- 直接影响芯片性能发挥。国产芯片（如龙芯、鲲鹏）以最大化硬件潜力。
- 研制、生产、维护，关键技术不受制于他人，能够可靠性、供货的稳定性。
- 操作系统、编译器、数据库、网络（软硬件）。

件生态战略的基石。

直接影响芯片性能发挥。国产芯片（如龙芯、鲲鹏）以最大化硬件潜力。

研制、生产、维护，关键技术不受制于他人，能够可靠性、供货的稳定性。

操作系统、编译器、数据库、网络（软硬件）。

域需要定制化编译技术。例如谷歌 XLA 编译器针对，自主编译器能快速适配国产 AI 芯片（如寒武纪）。

- 件生态战略的基石。
- 直接影响芯片性能发挥。国产芯片（如龙芯、鲲鹏）以最大化硬件潜力。
- 研制、生产、维护，关键技术不受制于他人，能够可靠性、供货的稳定性。
- 操作系统、编译器、数据库、网络（软硬件）。
- 域需要定制化编译技术。例如谷歌 XLA 编译器针对，自主编译器能快速适配国产 AI 芯片（如寒武纪

生态战略的基石。

直接影响芯片性能发挥。国产芯片（如龙芯、鲲鹏）最大化硬件潜力。

制、生产、维护，关键技术不受制于他人，能够自主性、供货的稳定性。

作系统、编译器、数据库、网络（软硬件）。

需要定制化编译技术。例如谷歌 XLA 编译器针对自主编译器能快速适配国产 AI 芯片（如寒武纪），成为创新基石，自主参与可主导标准制定。

原编译工具链的成熟。

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡ ▶ ↺ 🔍 ↻

生态战略的基石。

直接影响芯片性能发挥。国产芯片（如龙芯、鲲鹏）最大化硬件潜力。

制、生产、维护，关键技术不受制于他人，能够自主性、供货的稳定性。

作系统、编译器、数据库、网络（软硬件）。

需要定制化编译技术。例如谷歌 XLA 编译器针对自主编译器能快速适配国产 AI 芯片（如寒武纪），成为创新基石，自主参与可主导标准制定。

原编译工具链的成熟。

Figure 1. The effect of the number of trials on the mean accuracy of the responses. The error bars represent the standard error of the mean.

- 硬件生态战略的基石。
- 直接影响芯片性能发挥。国产芯片（如龙芯、鲲鹏）难以最大化硬件潜力。
- 自主研发、生产、维护，关键技术不受制于他人，能够保障供应链的可靠性、供货的稳定性。
- 操作系统、编译器、数据库、网络（软硬件）。
- 特定领域需要定制化编译技术。例如谷歌 XLA 编译器针对特定硬件优化，自主编译器能快速适配国产 AI 芯片（如寒武纪）。自主编译器已成为创新基石，自主参与可主导标准制定。
- 开源编译工具链的成熟。
- 突破性能边界。英伟达 CUDA 编译器使其 GPU 在 AI 领域占据统治地位，助力国产数据库（如 OceanBase）突破性能瓶颈。

- Chrome V8 Javascript 漏洞

CVE-2025-2137	CNA: Chrome
Out of bounds read in V8 in Google Chrome prior to 134.0.6998.88 allowed a remote attacker to perform out of bounds memory access via a crafted HTML page. (Chromium security...	
Show more	
CVE-2025-2135	CNA: Chrome
Type Confusion in V8 in Google Chrome prior to 134.0.6998.88 allowed a remote attacker to potentially exploit heap corruption via a crafted HTML page. (Chromium security severity: High)	
CVE-2025-1920	CNA: Chrome
Type Confusion in V8 in Google Chrome prior to 134.0.6998.88 allowed a remote attacker to potentially exploit heap corruption via a crafted HTML page. (Chromium security severity: High)	
CVE-2025-1914	CNA: Chrome
Out of bounds read in V8 in Google Chrome prior to 134.0.6998.35 allowed a remote attacker to perform out of bounds memory access via a crafted HTML page. (Chromium security...	
Show more	
CVE-2025-0999	CNA: Chrome
Heap buffer overflow in V8 in Google Chrome prior to 133.0.6943.126 allowed a remote attacker to potentially exploit heap corruption via a crafted HTML page. (Chromium security severity: High)	
CVE-2025-0995	CNA: Chrome
Use after free in V8 in Google Chrome prior to 133.0.6943.98 allowed a remote attacker to potentially exploit heap corruption via a crafted HTML page. (Chromium security severity: High)	

1. 植入非官方分发的 Xcode 安装包（主要针对中国）
 2. iOS 应用时注入以下行为：
 2.1 注入 C2 模块，绕过 App Store 静态检测
 2.2 将设备信息（UDID、iOS 版本）发送至 C2 服务器（IP：
 192.168.1.100）
 2.3 通过 C2 服务器下载乐、滴滴出行等 76 款热门应用被感染，累计下
 载量达 100 万
 2.4 通过 C2 服务器下载 Xcode（因国内下载速度慢）

-

1 绪论

为什么要学习编译原理?
为什么需要编译程序?
编译程序的作用与构成

② 语言与文法

③ 词法分析与正则文法

制硬件，编程效率极低且完全依赖硬件架构。过电路设计支持尽可能多的指令，包括但不限于各数等)。

过多条指令来实现一条复杂指令的工作，精简电路指令，使得程序的编写和阅读变得相对容易

码

处理一些复杂操作

◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡ 🔍 ↺

THE UNIVERSITY OF CHICAGO

- 沟通的桥梁作用
- 条件语句（如 switch-case）转换为底层跳转指令；
 - 类型检查（如 Java 强类型），避免运行异常；
 - 垃圾回收（如 Java 语言）隐藏手动内存操作的复杂性；
 - $b + c$ 转换为多条机器指令，自动处理数据移动和 ALU 操作。
- SQL (Structured Query Language)
- 数据库的声明式语言，支持数据定义（DDL）、操作（DML）和数据控制（DCL）。
- LaTeX (L^AT_EX)
- 排版语言，通过底层指令精确控制版面、数学公式和参考文献。
- HTML (Hypertext Markup Language)
- 标记语言，通过标签系统定义文本、图像等元素的结构。
- DSL (Domain-Specific Language)
- 领域特定语言，为特定领域设计的语言，是构建领域特定 DSL 的基石。

匹配语法实现字符串搜索、替换和验证，广泛
言，通过硬件描述逻辑实现芯片和 FPGA 的作
矩阵运算和仿真工具箱，用于工程建模与算法
量子比特操作和量子算法，与经典计算逻辑协

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡

1 绪论

为什么要学习编译原理？

② 语言与文法

③ 词法分析与正则文法

编译程序的作用

- 编译编译，顾名思义，有编有译，一编一译。
- 译——翻译：按照一定的规则把源翻译成另一种表示形式，如 AST、控制流图、三地址码、汇编代码等。
- 编——编制、整编：按照一定的顺序将翻译好的代码组织起来，填写符号表、间接跳转表，增加必要的进入与退出操作。
 - 用户程序的实际入口并非直接指向 main 函数，而是由编译器生成的启动代码（如 `_start`）处理初始化和清理工作。
 - GCC 在 x86 平台的 linux 系统上的操作：
 - 初始化堆栈和寄存器：设置栈指针（`rsp`）、基指针（`rbp`），确保内存布局符合 ABI 规范。
 - 处理命令行参数与环境变量：解析 `argc`、`argv` 和 `envp`，将其传递给 main 函数。
 - 全局构造与初始化：调用 `.init` 段代码（如 `_init` 函数）和执行全局 / 静态对象的构造函数（C++），通过 `.init_array` 或 `.ctors` 段中的函数指针列表。
 - C 运行时库（CRT）初始化：初始化标准 I/O 流（`stdin`、`stdout`、`stderr`）、设置堆管理（如 `malloc` 的元数据）、注册 `atexit` 和 `at_quick_exit` 回调函数。

理源代码中的预处理指令（如 `#include`、
整理，“译”即翻译。分析高级语言程序并翻译
码、标记等；
指令翻译为二进制机器代码，计算相对跳转位
多个二进制目标文件进行关联，处理外部函数跳
成可执行文件。

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡ 🔍 ↺

预处理后的源代码分解为一个个词法单元

； 2) 识别关键字、标识符、常量、运算符等。

根据语言的语法规则（文法），将词法单元序列解析为抽象语法树（Abstract Syntax Tree, AST）。

构建语法树来表示程序的结构。

3) 语义分析（Semantic Analysis）：检查程序的语义是否正确，例如类型检查、变量声明与使用的一致性、操作符与操作数的类型匹配、构建符号表（Symbol Table），记录变量、函数、类型等信息。

4) 生成中间表示（Intermediate Representation, IR）：将AST转换为一种与具体硬件无关的中间表示形式，如三地址码、控制流图（CFG）等。

5) 生成目标代码（Target Code Generation）：将中间表示转换为特定目标架构的机器码或字节码。

6) 优化（Optimization）：对生成的目标代码进行优化，以提高程序的执行效率。优化技术包括常量传播、死代码消除、基本块重组、局部变量优化、全局变量优化、控制流图优化、寄存器分配等。

7) 链接（Linking）：将目标代码与系统库、其他目标文件链接，生成可执行文件。

8) 运行（Execution）：将可执行文件加载到内存中，由操作系统调度执行。

9) 调试（Debugging）：提供调试工具，帮助开发者定位程序中的错误。

10) 性能分析（Performance Analysis）：对程序的性能进行分析和优化，以提高其运行效率。

11) 安全分析（Security Analysis）：对程序进行安全分析，检测潜在的安全漏洞。

12) 其他工具：包括代码覆盖率分析、静态分析、动态分析、性能剖析器等。

编译器的核心任务是生成目标代码，并对其进行优化。编译器的设计通常遵循模块化原则，将不同的功能模块分离开来，以便于维护和扩展。

编译器的实现通常涉及大量的算法和数据结构，如词法分析器、解析器、语义分析器、中间表示生成器、目标代码生成器等。编译器的设计目标是提高程序的执行效率，同时保证程序的正确性和可移植性。

编译器的设计是一个复杂的过程，需要综合考虑性能、可移植性、易用性等因素。随着计算机技术的不断发展，编译器的设计也在不断进步，出现了许多新的编译技术和工具，如即时编译器（JIT）、解释型编译器（Interpreter）、编译时优化（Compile-time Optimization）等。

编译器的设计是计算机科学中的一个重要领域，也是计算机工程中的一个核心问题。通过深入研究和实践，我们可以更好地理解和掌握编译器的设计原理和方法，为开发高性能、高可靠性的软件系统提供有力支持。

- 1 词法分析 (Lexical Analysis): 将预处理后的源代码分解为一个个词法单元 (Token)。
 - 过程: 1) 忽略空白字符和注释; 2) 识别关键字、标识符、常量、运算符等。
 - 输出: 词法单元序列。
- 2 语法分析 (Syntax Analysis): 根据语言的语法规则 (文法), 将词法单元序列解析为抽象语法树 (Abstract Syntax Tree, AST)。
 - 过程: 检查语法是否正确, 构建语法树来表示程序的结构。
 - 输出: 抽象语法树。
- 3 语义分析阶段 (Semantic Analysis): 检查程序的语义是否正确, 例如类型检查、变量作用域分析等。
 - 过程: 检查变量是否声明, 检查类型是否匹配, 构建符号表 (Symbol Table), 记录变量、函数等的定义和属性。
 - 经过语义检查和修饰的抽象语法树。

de Generation)：将抽象语法树转换为中间代码。中间代码是一种与硬件无关的低级表示形式，用于中间代码生成。

中间代码进行优化，提高程序的运行效率。

式消除、循环优化等。

) 和后端优化 (与硬件相关)。

)：将优化后的中间代码转换为目标代码 (机器码、汇编代码、Java 字节码、Python 字节码等)。

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡ 🔍 ↺

② 语言与文法

高级语言的构成 形式化文法

③ 词法分析与正则文法

高级语言的构成

编程语言通常由以下几个核心部分构成：

- ① 语法 (Syntax)：语法是编程语言的结构规则，定义了程序如何书写。它是语言的“形式”，规定了哪些字符、符号和结构是合法的。

例

C 语言使用分号 (;) 表示一般语句的结束，使用 {} 来表示语句块 (序列)。

- ② 语义 (Semantics)：语义是编程语言的“意义”，定义了程序的行为。它描述了程序如何被执行，以及每条语句的具体含义。

例

$a=1$ 在 C 语言中的意义是将 1 赋值给 a ，而在 Pascal 语言中则是判断 a 与 1 是否相等。

是编程语言提供的一组预定义函数和库函数，包括输入输出、内存管理、数学运算等。在 C 语言中，使用 `printf` 函数来实现格式化输出。

`printf` 函数原型如下：

```
int printf(const char *format, ...);
```

该函数能进行处理。在 C 语言中，网络通信通常依赖于操作系统提供的网络通信模块，而 C 语言则需要根据操作系统调用不同操作系统的网络通信模块。

例	
---	--

- C 语言在 `stdio.h` 头文件中提供了 `printf` 函数来实现格式化输出。
- Java 语言则需要使用 `System.out.printf`。
- Python 则通过格式化字符串 f-string 功能进行处理。
- Python 封装了统一的 `socket` 接口来处理网络通信，而 C 语言则需要根据操作系统不同使用 `socket` 或者 `winsock` 来支持调用不同操作系统的网络通信模块。



语法与语言

- 在人类社会中，**语言**是一种社会现象，是人类用于交流思想、表达情感和传递信息的符号系统。**语法**是语言中对词汇、句子结构等规则的总结。
- 在形式语言理论中，**文法 (Grammar)** 和 **语言 (Language)** 是两个核心概念，它们之间的关系概括为：**文法是语言的生成规则，而语言是文法生成的字符串集合**。
- 基于乔姆斯基的层级理论 (Chomsky Hierarchy)，按照生成能力从**强**到**弱**分成四类：

文法类型	文法名称	生成的语言	对应的自动机	例子
0 型文法	无限制文法	递归可枚举语言	图灵机	$a^n b^n c^n$
1 型文法	上下文相关文法	上下文有关语言	线性有界自动机	$a^n b^n c^n$
2 型文法	上下文无关文法	上下文无关语言	下推自动机	$a^n b^n$
3 型文法	正则文法	正则语言	有限状态自动机	$a^* b^*$

文法的形式化定义

文法的形式化定义为一个四元组 G :

$$G = (V_T, V_N, S, P).$$

其中,

- V_T 是终结符号集合, 可以理解为最终字符, 其中有一个特殊符号 ε , 用于表示空;
- V_N 是非终结符号集合, 非终结符号也称**文法符号**;
- S 是初始符号;
- P 是产生式/推导式的集合。

文法的产生式

- 产生式是形如 $\alpha A \rightarrow B\gamma$ 的式子。
- 注意，产生式的左侧必须要有一个非终结符。**
- 语言推导的过程就是从初始符号开始，每次用一个产生式右边字符串替换左边字符串，最终生成期望字符串的过程。
- 不同类型语法对产生式的约束如下：
 - 无限制文法：没有约束；
 - 上下文相关文法：**左边**的符号串长度不超过**右边**的符号串长度，如 $\alpha A\beta \rightarrow \alpha\gamma\beta$ ；
 - 上下文无关文法**：**产生式左边只有一个非终结符**，右边可以是任意字符串，如 $A \rightarrow aBa$ ；
 - 正则文法**的产生式规则必须满足以下两种形式之一：
 - 右正则形式：形如 $A \rightarrow aB$ 或 $A \rightarrow a$ ，其中 A 和 B 是非终结符， a 是终结符或者 ε 。
 - 左正则形式：形如 $A \rightarrow Ba$ 或 $A \rightarrow a$ ，其中 A 和 B 是非终结符， a 是终结符或者 ε 。

个阶段，其目的是将源代码分解为一个个词法单元（如空白字符和注释）。
它是编译器的核心组件之一，它为后续的语法分析和语义分析提供基础。
主要功能包括：
1. 提高可维护性、复用性；
2. 检测、合并重复标识符节约存储空间；
3. 将源代码抽象为词法单元序列，使得语法分析器可以专注于处理字符级别的细节。

- 等级的细节。

- ① 基于首字符的状态机的手工构造，例如：
 - 在一般模式下遇到" 进入字符串识别模式，直到遇到一个非转义的"，如"I said,\"Hello!\";
 - 在一般模式下遇到 0-9 进入数字常量识别模式，可处理带前缀和后缀的数字常量，如 0x1234,0777,0b1010,100ULL,0.345L;
 - 在一般模式下遇到运算符，判断是否需要处理运算符的合并，如 ==,+=,<=;
 - 在一般模式下遇到 #,/,/* 等进入注释处理模式;
 - 处理标识符并记录生成符号表;
 - 记录行首空格，例如 **Python** 语言以此作为语句块的识别;
 - 记录每个 token 出现在源代码中的位置，便于错误处理和提示;
- ② 基于正则文法/正则表达式的自动生成;

形式化规则。每个正则文法都对应一个正则语言。
一种**紧凑表示方法**。每个正则表达式都对应一个正则语言。
转换。例如，任何正则文法都可以转换为等价的正则表达式。
中正则语言的基础，广泛应用于词法分析、编译原理等。

- 形式化规则。每个正则文法都对应一个正则语言。
一种**紧凑表示方法**。每个正则表达式都对应一个正则语言。
转换。例如，任何正则文法都可以转换为等价的正则表达式。
中正则语言的基础，广泛应用于词法分析、编译原理等。

的表达式，它可以表示正则语言。正则表

。

ab 表示 a 后跟 b)。

| 表示，如 $a|b$ 表示 a 或 b) ²。

表示，如 a^* 表示零个或多个 a)。

表示，如 $a?$ 表示零个或一个 a)。

示任意一个小写字母)。

◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡ ≡ ≡ ≡ ↺ 🔍 ↻

例

- a^* 表示零个或多个 a 。
- $(a|b)^*$ 表示任意长度的 a 和 b 的组合。
- ab^*c 表示 a 后跟零个或多个 b ，再跟一个 c 。
- $a^+ = aa^*$ ，表示一个或多个 a 。

正则文法或正则表达式定义的语言。正则语言

动机（如 DFA 或 NFA）来识别。

、补集、连接和闭包等操作下是封闭的。

（如是否为空、是否有限等）是可判定的。

- 有限性：正则语言可以用有限自动机（如 DFA 或 NFA）来识别。
- 封闭性：正则语言在并集、交集、补集、连接和闭包等操作下是封闭的。
- 可判定性：正则语言的许多性质（如是否为空、是否有限等）是可判定的。

定义的语言不包含“ abb ”子串。

定义的语言不包含“ abb ”的子序列。

定义的语言中所有串恰好有偶数个 a 和偶数个 b 。

定义的语言中所有串恰好有奇数个 a 和偶数个 b 。

- 定义的语言不包含“ abb ”子串。
- 定义的语言不包含“ abb ”的子序列。
- 定义的语言中所有串恰好有偶数个 a 和偶数个 b 。
- 定义的语言中所有串恰好有奇数个 a 和偶数个 b 。

例

- $L \cup M = \{A, B, C, D, 1, 2, 3\}$
- $LM = \{A1, A2, A3, B1, B2, B3, C1, C2, C3, D1, D2, D3\}$
- $L^2 = \{AA, AB, AC, AD, BA, BB, BC, BD, CA, \dots DD\}$
- $L^* = \{\text{所有 ABCD 组成的串, 包含 } \varepsilon\}$
- $L^+ = L^* - \{\varepsilon\}$
- $L(L \cup M) = ?$
- $L(L \cup M)^* = ?$



- 1 绪论
- 2 语言与文法
- 3 词法分析与正则文法
NFA、DFA、子集构造法
- 4 语法分析与上下文无关文法

A) 由两位神经物理学家 McCulloch 和 Pitts 建立的**数学模型**；

输出信息和**有穷**数目的内部状态（状态：概括当前的输入信息就可以决定系统的后继行为的输入后，系统的内部状态也将发生改变）

- A) 由两位神经物理学家 McCulloch 和 Pitts 建立的**数学模型**；
- 输出信息和**有穷**数目的内部状态（状态：概括当前的输入信息就可以决定系统的后继行为的输入后，系统的内部状态也将发生改变）

式定义如下：

$(\Sigma, Q, q_0, \delta, F)$,

输入字符有哪些；

$= q'$ ，其需要两个输入 q, a , $q \in Q$ 为当前状态为处理输入后的状态；

A set of small navigation icons typically found in Beamer presentations, including symbols for back, forward, search, and other slide controls.

非确定性有穷状态自动机

非确定性有穷状态自动机 (NFA) 的形式化定义 (1) 如下:

$$M = (\Sigma, Q, q_0, \delta, F),$$

其中，

- Σ 是定义的字符集，即处理的输入字符有哪些；
- Q 是有限的状态集合；
- q_0 是初始状态；
- δ 是状态转移函数，形如 $\delta(q, a) = q'$ ，其需要两个输入 q, a ， $q \in Q$ 为当前状态， $a \in \Sigma \cup \{\epsilon\}$ 为当前输入， $q' \in Q$ 为处理输入后的状态；
- $F \subset Q$ 是接受状态集合；

[illegible]

- Σ 是定义的字符集，即处理的输入字符有哪些；
- Q 是有限的状态集合；
- q_0 是初始状态；
- δ 是状态转移函数，形如 $\delta(q, a) = \{s_1, s_2, \dots, s_k\}$ ，其需要两个输入 q, a ， $q \in Q$ 为当前状态， $a \in \Sigma$ 为当前输入， $\{s_1, s_2, \dots, s_k\} \subseteq Q$ 为处理输入后的状态；
- $F \subset Q$ 是接受状态集合；

消除不确定性的子集构造法

Algorithm 1: NFA 转 DFA

输入: NFA: $(Q, \Sigma, \delta, q_0, F)$

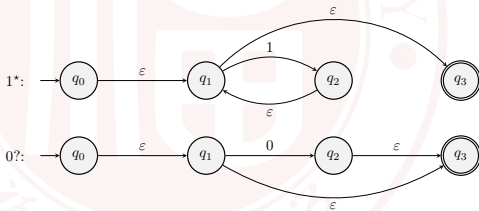
输出: DFA: $(Q', \Sigma, \delta', q'_0, F')$

```
1  $Q' \leftarrow \emptyset;$ 
2  $\delta' \leftarrow \emptyset;$ 
3  $q'_0 \leftarrow \varepsilon\text{-Closure}(\underline{q_0});$ 
4  $Q'.\text{add}(q'_0);$ 
5 创建未标记状态队列  $workList \leftarrow \{q'_0\};$ 
6 while  $workList$  非空 do
7   从  $workList$  中取出状态  $S;$ 
8   foreach 输入符号  $a \in \Sigma$  do
9      $S' \leftarrow \varepsilon\text{-Closure}(\underline{\text{Move}(S, a)});$ 
10    if  $S' \notin Q'$  then
11       $Q'.\text{add}(S');$ 
12       $workList.\text{add}(S');$ 
13     $\delta'[S, a] \leftarrow S';$ 
14  $F' = \{S \in Q' \mid S \cap F \neq \emptyset\};$ 
```

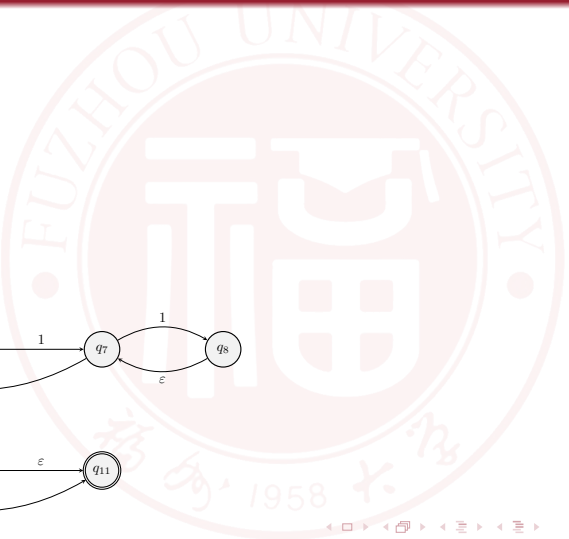
定义的语言不包含“100”子串，并构造其对应

① 写出正则表达式 $0^*1^*(01^+)^*0^?$;

- ① 写出正则表达式 $0^*1^*(01^+)^*0^?$;
- ② 依次构造 0^* 、 1^* 、 $(01^+)^*$ 、 $0^?$ 的 NFA;



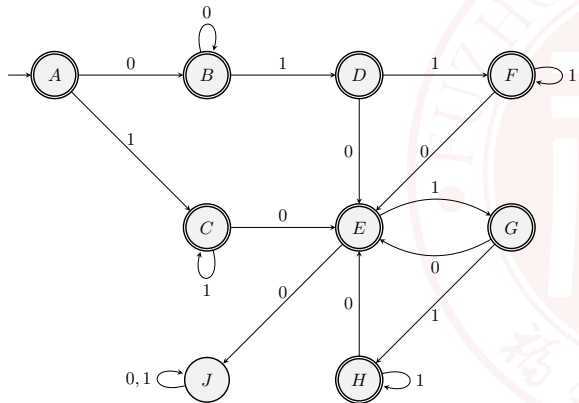
Journal of Management Inquiry 20(6) 798–814



1

综合示例（续）

④ NFA to DFA;



45 / 118

符合目标语言的语法规则，并将源代码转换为语法树)。

语法分析器生成的词法单元序列，确保程序的源代码生成提供基础。

去。

- 语法分析目的是验证源代码是否符合目标语言的语法规则，并将源代码转换为一种结构化的表示形式（如抽象语法树）。
- 语法分析器（Parser）通过分析词法分析器生成的词法单元序列，确保程序的语法正确性，并为后续的语义分析和代码生成提供基础。
- 语法分析的基础是上下文无关文法。

中文法
如下形式

$$N \rightarrow (\Sigma \cup N)^*,$$

仅允许存在一个非终结符。

上下文无关文法 $G = (N, \Sigma, P, S)$, 其中文法

- 产生式规则集 P , 每个规则都为如下形式

$$N \rightarrow (\Sigma \cup N)^*,$$

即上下文无关文法的产生式左侧仅允许存在一个非终结符。

用于识别和接受字符输入。
但多出一个称为栈（stack）的组件，且栈的规
定性的和非确定性的不能等价。

- 下推自动机，是一个抽象的机器用于识别和接受字符输入。
- 它和非确定型有限自动机类似，但多出一个称为栈（stack）的组件，且栈的规模是无限的。
- PDA 默认都是非确定性的，确定性的和非确定性的不能等价。

一个 PDA 可以被定义为六元组

$$M = (Q, \Sigma, \Gamma, \delta, q_0, F),$$

其中,

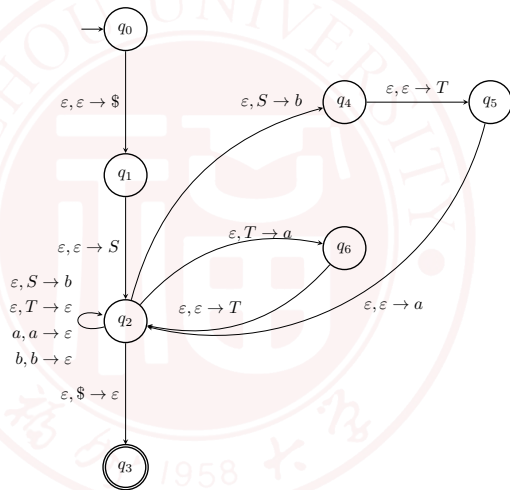
- Q 是所有状态的集合, 有限个;
- Σ 是输入字符的集合, 有限。特别地, $\Sigma_\epsilon = \epsilon \cup \Sigma$
- Γ 是栈字符的集合, 有限。特别地, $\Gamma_\epsilon = \{\epsilon\} \cup \Gamma$
- δ 是转移函数

$$\delta : Q \times \Sigma_\epsilon \times \Gamma_\epsilon \rightarrow Q \times \Gamma_\epsilon$$

根据输入和当前栈顶元素进行转移的同时替换栈顶元素特别地, 转移函数支持 ϵ 作为栈顶元素, 也支持将栈顶元素替换为 ϵ 。这两个操作一个是单纯压栈, 另一个是单纯弹栈 q_0, F 的意思和 NFA 中的一致

上下文无关文法与对应的 PDA

$$S \rightarrow aTb \mid b$$
$$TTa \mid \varepsilon$$



语法分析器的类型

自顶向下 (Top-Down)

- 从开始符号出发，寻找“匹配”的推导。
- 推导：根据文法的产生式规则，把串中出现的产生式的左部符号替换成右部。
- 从根开始，构造语法树。
- 具体实现：递归下降分析法、预测分析法。

自底向上 (Bottom-Up)

- 从输入串开始，逐步进行归约，直到文法的开始符号。
- 归约：根据文法的产生式规则，把串中出现的产生式的右部替换成左部符号。
- 从叶节点开始，构造语法树。
- 具体实现：算符优先分析法、LR 分析法。

④ 语法分析与上下文无关文法

自顶向下的语法分析

文法的额外处理

递归下降法

预测分析法

自底向上的语法分析

Top-Down 分析器挑战一：左递归

在自顶向下分析器寻找匹配的推导时，通常采用的是最左推导的形式，即每次选择**最左边**的非终结符进行替换。

按顺序尝试非终结符的不同产生式，有可能出现最左侧符号不断出现的无限循环的情况。

文法

$$expr \rightarrow expr + term \mid expr - term \mid term$$
$$term \rightarrow term * factor \mid term / factor \mid factor$$
$$factor \rightarrow digit \mid (expr)$$
$$digit \rightarrow 0 \mid 1 \mid \dots \mid 9$$

目标句子: $8+8+8$

$$expr \Rightarrow expr + term$$
$$\Rightarrow \text{expr} + \text{term} + \text{term}$$
$$\Rightarrow \text{expr} + \text{term} + \text{term}$$
$$\Rightarrow \text{expr} + \text{term} + \text{term} + \text{term}$$
$$\Rightarrow \text{expr} + \text{term} + \text{term} + \text{term} + \text{term}$$

Top-Down 分析器挑战一：左递归

左递归的两种形式：

直接左递归

$$expr \rightarrow expr + term \mid expr - term \mid term$$
$$term \rightarrow term * factor \mid term / factor \mid factor$$
$$factor \rightarrow digit \mid (expr)$$
$$digit \rightarrow 0 \mid 1 \mid \dots \mid 9$$

间接左递归

$$S \rightarrow AB$$
$$A \rightarrow TC$$
$$T \rightarrow S\gamma$$

某一个候选式匹配成功时，这种匹配可能是“暂时”的。

当存在共同前缀时，将导致回溯。

当要它去匹配输入串时，能够根据它所面临的候选去执行任务，并且此候选的工作结果应是确信无疑的。

-

提取左公因式

数量优先

例

改写文法 $S \rightarrow abcd \mid abce \mid abf$ 以便无回溯的自顶向下分析。

解

S 所有子式的最长左公因子为 ab ，引入 A 改写生成式得 $S \rightarrow abA, A \rightarrow cd \mid ce \mid f$ 。

再提取 A 的左公因子并引入 B 改写生成式得 $A \rightarrow cB \mid f, B \rightarrow d \mid e$ 。

最终文法为

$$S \rightarrow abA$$

$$A \rightarrow cB \mid f$$

$$B \rightarrow d \mid e$$

递归下降分析法

- 自顶向下的语法分析：从分析树的根结点开始，深度优先地创建分析树的各个结点。可以看作是寻找输入串的最左推导的过程。
- 一个递归下降分析程序由一组过程（函数）组成。
- 每个非终结符号有一个对应的过程，识别对应的语法单位。
- 当需要从某个非终结符出发进行展开（推导）时，就调用这个非终结符对应的子程序。
- 程序的执行从开始符号对应的过程开始，如果这个过程的过程体扫描了整个输入串，它就停止执行并宣布语法分析成功。
- 通用的递归下降分析技术可能需要回溯，即它可能要重复扫描输入。

递归下降分析法

典型过程

Algorithm 3: 处理非终结符号的典型过程

```
1 void A () {  
2   选择一个 A 的产生式,  $A \rightarrow X_1, X_2, \dots, X_k$ ;  
3   for  $i \leftarrow 1$  to  $k$  do  
4     if  $X_i$  是一个非终结符号 then  
5       调用过程  $X_i()$ ;  
6     else  
7       if  $X_i$  等于当前的输入符号  $a$  then  
8         继续读取下一个输入符号;  
9       else  
10        处理错误;  
11      end  
12 end
```

- $\forall \alpha \in (\mathbb{V}_T \cup \mathbb{V}_N)^+, \text{FIRST}(\alpha) = \{a | \alpha \Rightarrow^* a\beta, a \in \mathbb{V}_T, \beta \in (\mathbb{V}_T \cup \mathbb{V}_N)^*\}$
- 若 $\alpha \Rightarrow^* \varepsilon$ 合法, 则 $\varepsilon \in \text{FIRST}(\alpha)$

Page 10 of 10

首集 FIRST(X) 的算法

1 如果 X 是一个终结符号,

计算 FIRST 的算法

Algorithm 5: 计算语法符号 X 的串首集 $\text{FIRST}(X)$ 的算法

Input: 语法符号 X

Output: 串首集 $\text{FIRST}(X)$

- 1 如果 X 是一个终结符号, 则 $\text{FIRST}(X) = \{X\}$;

计算 FIRST 的算法

Algorithm 6: 计算语法符号 X 的串首集 $\text{FIRST}(X)$ 的算法

Input: 语法符号 X

Output: 串首集 $\text{FIRST}(X)$

- 1 如果 X 是一个终结符号, 则 $\text{FIRST}(X) = \{X\}$;
- 2 如果 X 是一个非终结符号,

计算 FIRST 的算法

Algorithm 7: 计算语法符号 X 的串首集 $\text{FIRST}(X)$ 的算法

Input: 语法符号 X

Output: 串首集 $\text{FIRST}(X)$

- 1 如果 X 是一个终结符号, 则 $\text{FIRST}(X) = \{X\}$;
 - 2 如果 X 是一个非终结符号, 则先令 $\text{FIRST}(X) \leftarrow \Phi$;
 - 3 然后选定一个产生式 $X \rightarrow Y_1 Y_2 \dots Y_k$, 其中 $k \geq 1$;
 - 4 **for** $i \leftarrow 1$ **to** k **do**
 - 5 如果 $a \in \text{FIRST}(Y_i)$ 且 $\forall j < i, \varepsilon \in \text{FIRST}(Y_j)$, 则
 $\text{FIRST}(X) \leftarrow \text{FIRST}(X) \cup \{a\}$;
 - 6 **end**
 - 7 如果 $\forall i \geq k, \varepsilon \in \text{FIRST}(Y_i)$, 则 $\text{FIRST}(X) \leftarrow \text{FIRST}(X) \cup \{\varepsilon\}$;
 - 8 依次处理 X 的所有生成式, 对所有可能的 $\text{FIRST}(X)$ 取并集;
-

- 对于非终结符号 A , $\text{FOLLOW}(A)$ 是所有在某些句型中紧跟在 A 右边的终结符号的集合。
- A 与 a 之间可能存在其他的非终结符, 但都会随着推导变成 ε 。
- 特别地, 如果 A 是某些句型最右的符号, 则 $\$ \in \text{FOLLOW}(A)$, 其中 $\$$ 表示输入的终止 (EOF)。

例

如果存在 $S \Rightarrow^* \alpha A a \beta$ 的推导, 则 $a \in \text{FOLLOW}(A)$ 。

计算 FOLLOW 的算法

Algorithm 8: 计算语法符号 A 的紧随集 $\text{FOLLOW}(A)$ 的算法

Input: 语法符号 A

Output: 紧随集 $\text{FOLLOW}(A)$

- 1 将 $\$$ 放到 $\text{FOLLOW}(S)$ 中;
 - 2 如果存在一个产生式 $A \rightarrow \alpha B \beta$, 则 $\text{FIRST}(\beta)$ 中除了 ϵ 意外的符号都在 $\text{FOLLOW}(B)$ 中;
 - 3 如果存在一个产生式 $A \rightarrow \alpha B$, 或存在 $A \rightarrow \alpha B \beta \wedge \epsilon \in \text{FIRST}(\beta)$, 则 $\text{FOLLOW}(A)$ 所有符号都在 $\text{FOLLOW}(B)$ 中;
 - 4 上述步骤重复应用, 知道没有任何新的终结符号可以加入任何非终结符的 FOLLOW 集合中。
-

LL(1) 文法

定义

- LL(1) 文法是一类特别的上下文无关文法。
 - 第一个 L 表示 “Left-to-Right”，即从左往右扫描；
 - 第二个 L 表示 “Leftmost”，即最左推导；
 - 1 表示每次推导只需要看一个输入符号就可以决定语法分析动作，即决定使用那个推导式。
- 左递归文法和二义文法都不是 LL(1) 文法。
- 虽然在为语言设计文法的时候要多加小心，LL(1) 文法已经足以描述大部分程序设计语言构造。
- LL(1) 文法可以将 ε 边作为最后的默认选择以解决 ε 的二义性（例如，if 语句的 else 匹配问题）。

表驱动的预测分析法

定义

- **预测分析法**是一种表驱动的方法，它由下推栈，预测分析表和控制程序组成，是一种确定性的下推自动机。
- 特征：不需要为每个非终结符编写递归下降过程，而是根据预测分析表构造一个不需要回溯的非递归下降语法分析器。
- 要求：文法是 LL(1) 的。只有对于 LL(1) 文法，可以对输入串进行有效的无回溯的自上而下分析。

- **预测分析表**是用于自顶向下语法分析（如 LL(1) 分析器）的核心工具，它通过非终结符和输入符号的匹配，指导分析器选择正确的产生式，避免回溯。
- 预测分析表是一个二维表格（矩阵）：
 - 行代表每一个非终结符；
 - 列代表每一个终结符（包括结束符号 \$）；
 - 每个单元格里填写对应的产生式，或者错误标记；
 - 单元格 (i, j) 的内容表示若在匹配第 i 行的非终结符时遇到第 j 列的终结符时，该确定性地选用的生成式。

预测分析表

Algorithm 9: 预测分析表构造算法

Input: $\mathbb{P}$, $\text{FIRST}(\cdot)$, $\text{FOLLOW}(\cdot)$

```
1 for  $A \rightarrow \alpha \in \mathbb{P}$  do
2   if  $a \in \text{FIRST}(\alpha)$  then
3      $M[A, a] \leftarrow (A \rightarrow \alpha)$ ;
4   end
5   if  $\varepsilon \in \text{FIRST}(\alpha)$  then
6     for  $b \in \text{FOLLOW}(A)$  do
7        $M[A, b] \leftarrow (A \rightarrow \alpha)$ ;
8     end
9   end
10 end
11 剩余空格打上错误标记;
12 如果一个格子里面有多于 1 条规则, 不是 LL(1) 文法, 返回错误;
```

写出文法的预测分析表:

$$S \rightarrow iEtSS' \mid a$$

$$S' \rightarrow eS \mid \varepsilon$$

$$E \rightarrow b$$

推论

如果冲突的是生成 ϵ 的产生式, 那么可以将 ϵ 产生式丢弃。

预测分析表

	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow id$			$F \rightarrow (E)$		

例

基于上面的预测分析表，尝试解析 $(id * id) + id - id * id$ 。



① 绪论

② 语言与文法

③ 词法分析与正则文法

④ 语法分析与上下文无关文法

上下文无关文法与下推自动机 (PDA)

自顶向下的语法分析

自底向上的语法分析

移入-规约框架

相关概念

LR 分析法基础

LR(0) 分析法

SLR 分析法

LR(1) 分析法

从(根节点) 方向构造分析树，可以看成
号S 的过程。
左导方式。
 右约方式（反向构造**最右推导**）。

- ## 定义 (归约, Reduce)

归约：先将一个子串与某规则的右部匹配，然后将该子串替换为该规则的左部。

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡ ≡ ≡ ↺ 🔍 ↻



短语、句柄

对于文法 $G(S)$, $\alpha\beta\delta$ 是 G 的一个句型。如果有 $S \Rightarrow_{rm}^* \alpha A \delta$, 且

- $A \Rightarrow_{rm}^+ \beta$, 则称 β 是句型 $\alpha\beta\delta$ 相对于 A 的短语。

短语、句柄

对于文法 $G(S)$, $\alpha\beta\delta$ 是 G 的一个句型。如果有 $S \Rightarrow_{rm}^* \alpha A \delta$, 且

- $A \Rightarrow_{rm}^+ \beta$, 则称 β 是句型 $\alpha\beta\delta$ 相对于 A 的短语。
- $A \Rightarrow_{rm} \beta$, 则称 β 是句型 $\alpha\beta\delta$ 相对于 A 的直接短语（简单短语）。

短语、句柄

对于文法 $G(S)$, $\alpha\beta\delta$ 是 G 的一个句型。如果有 $S \Rightarrow_{rm}^* \alpha A \delta$, 且

- $A \Rightarrow_{rm}^+ \beta$, 则称 β 是句型 $\alpha\beta\delta$ 相对于 A 的短语。
- $A \Rightarrow_{rm} \beta$, 则称 β 是句型 $\alpha\beta\delta$ 相对于 A 的直接短语（简单短语）。
- 一个句型的最左直接短语称为该句型的句柄（Handle）。

向上的分析中，总是采用最左归约的方式，因此是最右推导的逆过程，则最右推导相应地称为规范句型。

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡

[illegible]

- 例

- ② 活前缀：文法 $G(S)$ 的一个规范句型 $\alpha\beta$ ，其中 β 为终结符号串。若 α 不含句柄之后的任何符号，则称 α 为活前缀（可行前缀）。
- ③ 如果 α 是含有句柄的活前缀，则称 α 为可归前缀。
- ④ 可归前缀即最长的活前缀。

- 移入/归约冲突：即使知道了所有栈内符号和后续的 k 个符号，也无法判断应该将新元素移入栈还是将栈中已有元素进行归约。

if 语句的抽象文法

$$S \rightarrow iEtSeS \mid iEtS \mid o$$

当栈内元素为 $iEt iEt S$ ，且下个元素为 e 时，应当将栈内元素先进行归约，还是移入 e 等待下一个 S 出现后再归约呢？

归约/归约冲突

有些文法不能使用移入-归约分析框架进行语法分析。它们会产生如下冲突：

- 移入/归约冲突：即使知道了所有栈内符号和后续的 k 个符号，也无法判断应该将新元素移入栈还是将栈中已有元素进行归约。
- 归约/归约冲突：即使知道了所有栈内符号和后续的 k 个符号，用哪个产生式归约。

例

假设有文法

$$S \rightarrow P;$$
$$S \rightarrow E := E;$$
$$P \rightarrow i(L)$$
$$L \rightarrow L, i \mid i$$
$$E \rightarrow i(D)$$
$$D \rightarrow D, i \mid i$$

当前栈中元素依次为 $i (i$ 。应当如何规约？

- 如果 i 对应的符号表示一个过程，则应该调用 $L \rightarrow i$ 进行归约；
- 如果 i 对应为一个数组，则应该调用 $D \rightarrow i$ 来归约；

问：看后面 $:=$ 不就可以决定了吗？

LR(k) 文法

- **LR(k) 文法**是一类特别的上下文无关文法。
 - L 表示 “Left-to-Right”，即从左往右扫描；
 - R 表示 “Rightmost”，即最右推导；
 - k 表示每次推导需要看 k 个输入符号 来决定语法分析动作，即决定什么时候归约？用哪个产生式归约？
- LR 文法的优势：
 - ① 语言的通用性：几乎所有语言都可以使用 LR 文法来定义；
 - ② 分析的高效性：最通用的无回溯移入归约分析法；
 - ③ 错误发现早：可以尽可能早的检测到错误；
 - ④ 支持的语言是 LL(1) 语言的真超集。
- LR 分析器的缺点：实现起来很复杂。

力作。
容易
(如

- $k = 0$ ，仅依赖**当前状态**决定动作。
- 构造简单，但分析能力有限，**容易产生冲突**。
- 仅能处理极简单的无冲突文法（如无二义性的规则文法）。

LR 分析器家族

① LR(0) 分析器

- $k = 0$ ，仅依赖当前状态决定动作。
- 构造简单，但分析能力有限，容易产生冲突。
- 仅能处理极简单的无冲突文法（如无二义性的规则文法）。

② SLR(1)（简单 LR）分析器

- 在 LR(0) 的基础上，利用 FOLLOW 集缩小归约范围，解决部分冲突。
- 向前看 1 个符号，但仅在归约时检查 FOLLOW 集。
- 能处理部分 LR(0) 无法解决的冲突，但对某些文法仍存在局限性。

LR 分析器家族

① LR(0) 分析器

- $k = 0$, 仅依赖当前状态决定动作。
- 构造简单, 但分析能力有限, 容易产生冲突。
- 仅能处理极简单的无冲突文法 (如无二义性的规则文法)。

② SLR(1) (简单 LR) 分析器

- 在 LR(0) 的基础上，利用 FOLLOW 集缩小归约范围，**解决部分冲突**。
- 向前看 1 个符号，但仅在归约时检查 FOLLOW 集。
- 能处理部分 LR(0) 无法解决的冲突，但对某些文法仍存在局限性。

③ 规范 LR(1) 分析器

- 最强大的 LR 分析器，精确跟踪每个项的向前看符号。
- 状态数远多于 LALR(1)，但能处理所有 LR(1) 文法。
- 用于需要处理复杂冲突的文法，但实现成本较高。

LR 分析器家族

① LR(0) 分析器

- $k = 0$ ，仅依赖**当前状态**决定动作。
- 构造简单，但分析能力有限，**容易产生冲突**。
- 仅能处理极简单的无冲突文法（如无二义性的规则文法）。

② SLR(1)（简单 LR）分析器

- 在 LR(0) 的基础上，利用 FOLLOW 集缩小归约范围，**解决部分冲突**。
- 向前看 1 个符号，但仅在归约时检查 FOLLOW 集。
- 能处理部分 LR(0) 无法解决的冲突，但对某些文法仍存在局限性。

③ 规范 LR(1) 分析器

- 最强大的 LR 分析器，精确跟踪每个项的向前看符号。
- 状态数远多于 LALR(1)，但能**处理所有 LR(1) 文法**。
- 用于需要处理复杂冲突的文法，但实现成本较高。

④ LALR(1)（向前看 LR）分析器

- 基于 LR(1) 项集，但通过合并同心项集减少状态数量。
- 状态数与 SLR(1) 接近，但**处理能力更强**。
- 大多数编程语言的**实际文法**，广泛用于编译器工具（如 Yacc/Bison）。

LR 家族各自优劣势

分析器类型	状态数	优势	劣势
LR(0)	最少	构造简单， 状态数少	极易产生移进-归约 或归约-归约冲突
SLR(1)	与 LR(0) 相同	冲突处理优于 LR(0)， 实现简单	对 FOLLOW 集外的冲突 仍无法解决
LALR(1)	接近 SLR(1)	状态数少且能力强， 平衡性能与复杂度	可能无法处理某些 规范 LR(1) 文法
规范 LR(1)	最多 ⁵	无冲突，能力最强	状态数庞大 构造和存储成本极高

⁵指数级增长

LR 家族具体分析示例

例

假设文法:

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid id$$

分析器类型	能否无冲突分析	状态数	冲突类型
LR(0)	不能	~10	移进 vs 归约
SLR(1)	能	~10	无冲突
LALR(1)	能	~12	无冲突
规范 LR(1)	能	~30	无冲突

号 a 时，应采取的动作。

时的下一状态。

GOTO[s, X]: 状态 s 面对文法符号 X 时的下一状态。

LR(0) 分析法概述

- 构成要件
 - 项 (item)
 - 项集闭包 (closure)
 - GOTO 函数
- 构建流程
 - ① 拓广文法
 - ② 构造项集族
 - ③ 构建 LR(0) 分析表
- 分析算法
 - 分析栈结构
 - 分析步骤
- 局限
 - 移进-归约冲突
 - 归约-归约冲突

Figure 1

文法 G 的一个 **LR (0) 项** (简称为项) 是 G 的一个产生式再加上一个式体中某处的点。

$$A \rightarrow XYZ \Rightarrow$$

例

$S \rightarrow \cdot aBc$ (未开始识别)
 $S \rightarrow a \cdot Bc$ (已识别 a, 待识别 Bc)
 $S \rightarrow aB \cdot c$ (已识别 aB, 待识别 c)
 $S \rightarrow aBc \cdot$ (已完成识别)

规范 LR(0) 项集族

规范 LR(0) 项集族 (canonical LR(0) collection) 是**项集的集合**，是构建**确定性 LR(0) 自动机**的基础。

构造规范 LR(0) 项集族的主要步骤：

- ① 增广文法（拓广文法，augmented grammar）目的是保证文法有唯一的接受状态；
- ② 通过生成式构造项；
- ③ 闭包（CLOSURE）计算；
- ④ GOTO 计算；
- ⑤ 计算规范 LR(0) 项集族。

92 / 118

项集闭包

例

假设初始项集 $I = \{S' \rightarrow \cdot S\}$ ，且存在 $S \rightarrow abc$ ，则闭包扩展后有

$$\text{CLOSURE}(I) = \{S' \rightarrow \cdot S, S \rightarrow \cdot abc\}$$

- 如果有一个点在最左端的 B 的产生式被加入项集，则其余 B 的产生式也会被加入项集⁶。
- 内核项与非内核项：
 - 内核项：初始项 $S' \rightarrow \cdot S$ 和其他点不在左端的所有项；
 - 非内核项：除了初始项 $S' \rightarrow \cdot S$ 以外其他点在左端的项。

内核项有进度信息需要存储，非内核项无需记录可以通过闭包运算重新获得。

⁶无需列出所有产生式，记录一下哪个文法符号即可

GOTO 函数

$$\text{GOTO}(\mathbf{I}, X) := \text{CLOSURE}(\{A \rightarrow \alpha X \cdot \beta \mid A \rightarrow \alpha \cdot X \beta \in \mathbf{I}\})$$

例

输入项集 $\mathbf{I} = \{S \rightarrow a \cdot B, B \rightarrow \cdot c, B \rightarrow \cdot d\}$, 面临符号 B , 则

$$\text{GOTO}(\mathbf{I}, B) = \{S \rightarrow aB \cdot\}$$

构造规范 LR(0) 项集族

Algorithm 11: 规范 LR(0) 项集族的计算

```
1  $\mathcal{C} \leftarrow \{\text{CLOSURE}(\{[S' \rightarrow \cdot S]\})\};$ 
2 repeat
3   for  $\mathbf{I} \in \mathcal{C}$  do
4     for  $X \in \mathbb{V}_N \cup \mathbb{V}_T$  do
5       if  $\text{GOTO}(\mathbf{I}, X) \neq \emptyset \wedge \text{GOTO}(\mathbf{I}, X) \notin \mathcal{C}$  then
6          $\mathcal{C} \leftarrow \mathcal{C} \cup \text{GOTO}(\mathbf{I}, X);$ 
7       end
8     end
9   end
10 until 没有新的项集被加入  $\mathcal{C}$ ;
```

LR(0) 自动机的作用

规范归约过程中

- 栈内的符号串和扫描剩下的输入符号串构成了一个规范句型；
- 栈内如果出现句柄，句柄一定在栈的顶部；
- 一旦栈顶出现可归约串 (句柄)，则进行归约。
- 识别了句柄的一部分（即规范句型的活前缀）就相当于识别了当前规范句型的左起部分

因此，我们可以得出一个指导 LR 分析的目标——保证栈中总是**活前缀**。对于文法 G，可构造一个 DFA 来识别 G 的所有活前缀。

LR(0) 自动机

LR(0) 自动机是一个确定性有穷状态自动机 (DFA)。

五元组的对应关系:

构成要件	DFA	LR(0) 自动机
字符集	Σ	文法符号 (含终结符和非终结符)
状态	q	一个项集的闭包
有限的状态集合	Q	规范 LR(0) 项集族
状态转移函数	$\delta(q, a)$	GOTO 函数
起始状态	q_0	$\text{CLOSURE}(\{[S' \rightarrow \cdot S]\})$
接受状态集合	F	包含项 $[S' \rightarrow S \cdot]$ 的项集对应 \$ 输入时的状态

推论

- 有点在最右侧的状态时应当调用归约过程；

推论

- 当前状态所对应项集闭包中，含有点在最右侧的状态时应当调用归约过程；
- 调用完归约过程后，需要将归约得到的文法符号视作输入继续进行转移；

有点在最右侧的状态时应当调用归约过程；
得到的文法符号视作输入继续进行转移；
归约；

推论

- 当前状态所对应项集闭包中，含有点在最右侧的状态时应当调用归约过程；
- 调用完归约过程后，需要将归约得到的文法符号视作输入继续进行转移；
- 回到第一步，继续判断是否可以归约；

推论

- 当前状态所对应项集闭包中，含有点在最右侧的状态时应当调用归约过程；
- 调用完归约过程后，需要将归约得到的文法符号视作输入继续进行转移；
- 回到第一步，继续判断是否可以归约；
- 状态栈描述了完整的转移流程，可以据此还原符号栈，可以减少保存一个符号栈，节约内存；

LR 分析器的语法分析表由两个部分构成：语法分析动作函数 ACTION 和转换函数 GOTO。

语法分析动作函数 ACTION

语法分析动作函数 ACTION[i, a], 其中 i 为状态和 a 为当前输入 (或终结符 \$), 其值有四种形式:

- 1 移入 s_j 。移入状态 j (同时移入符号 a)；
- 2 归约 r_j 。假设第 j 条规则为 $A \rightarrow \beta$ ，则将栈顶的 β 归约为 A ；
- 3 接受。接受输入串，完成语法分析。
- 4 报错。在输入中发现一个错误并执行纠正动作。

转换函数 $GOTO[l_i, A]$ ，其中 i 为状态编号和 A 为非终结符。若 $GOTO[l_i, A] = l_j$ ，则在当前状态为 i ，归约得到非终结符 A 时，移入与 A 对应的状态 j 。

授课教师：柯嵩宇

LR(0) 分析表

如果基于某文法构建的所有 LR(0) 项集，要么只有移入项目，要么只有~~一个~~归约项目，则可以构建 LR(0) 分析表。同时有：

- 该文法称为 LR(0) 文法；
- 在分析时，不需要看后续输入即可确定动作；
- 分析表中，归约项目无需考虑后继。

LR(0) 分析表的构造

- ① 构造 G' 的规范 LR(0) 项集族 $\mathcal{C} = \{I_0, I_1, \dots, I_n\}$ 。
- ② 令 I_i 对应状态 i , 则 ACTION 表和 GOTO 表构造如下:
 - $(A \rightarrow \alpha \cdot a\beta \in I_i \wedge \text{GOTO}(I_i, a) = I_j) \longrightarrow \text{ACTION}[i, a] = s_j$;
 - $(A \rightarrow \alpha \cdot B\beta \in I_i \wedge \text{GOTO}(I_i, B) = I_j) \longrightarrow \text{GOTO}[i, B] = s_j$;
 - $(A \rightarrow \alpha \cdot \in I_i \wedge A \neq S') \implies \forall a \in \mathbb{V}_T \cup \{\$\} \text{ACTION}[i, a] = r_j$, 其中 j 是产生式 $A \rightarrow \alpha$ 的编号;
 - $(S' \rightarrow S \cdot \in I_i) \implies \text{ACTION}[i, \$] = \text{acc}$;
 - 没有定义的所有条目都设置为 “error”。

分析动作冲突，那么给定的文法就称为**LR(0)**文法。

LR(0)文法有限，没有太大的实用价值。

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡

SLR 分析法的基本思路

已知项集 I 有 m 个移进项目和 n 个归约项目分别为

$$\begin{array}{ll} A_1 \rightarrow \alpha_1 \cdot a_1 \beta_1 & B_1 \rightarrow \gamma_1 \cdot \\ A_2 \rightarrow \alpha_2 \cdot a_2 \beta_2 & B_2 \rightarrow \gamma_2 \cdot \\ \vdots & \vdots \\ A_m \rightarrow \alpha_m \cdot a_m \beta_m & B_n \rightarrow \gamma_n \cdot \end{array}$$

如果集合 $\{a_1, a_2, \dots, a_m\}$ 和 $\text{FOLLOW}(B_1), \text{FOLLOW}(B_2), \dots, \text{FOLLOW}(B_n)$ 两两不相交, 则项目集 I 中的冲突可以按以下原则解决:

- 若 $a \in \{a_1, a_2, \dots, a_m\}$, 则移进 a ;
- 若 $a \in \text{FOLLOW}(B_i)$, 则用产生式 $B_i \rightarrow \gamma_i$ 归约;
- 其他情况报错。

SLR 分析表

- ① 构造 G' 的规范 LR(0) 项集族 $\mathcal{C} = \{I_0, I_1, \dots, I_n\}$ 。
- ② 令 I_i 对应状态 i , 则 ACTION 表和 GOTO 表构造如下:
 - $(A \rightarrow \alpha \cdot a\beta \in I_i \wedge \text{GOTO}(I_i, a) = I_j) \rightarrow \text{ACTION}[i, a] = s_j$;
 - $(A \rightarrow \alpha \cdot B\beta \in I_i \wedge \text{GOTO}(I_i, B) = I_j) \rightarrow \text{GOTO}[i, B] = s_j$;
 - $(A \rightarrow \alpha \cdot \in I_i \wedge A \neq S') \rightarrow \forall a \in \text{FOLLOW}(A) : \text{ACTION}[i, a] = r_j$, 其中 j 是产生式 $A \rightarrow \alpha$ 的编号;
 - $(S' \rightarrow S \cdot \in I_i) \rightarrow \text{ACTION}[i, \$] = \text{acc}$;
 - 没有定义的所有条目都设置为 “error”。

SLR 冲突

回顾一下：SLR 分析其中我提到的信息——

如果集合 $\{a_1, a_2, \dots, a_m\}$ 和 $\text{FOLLOW}(B_1), \text{FOLLOW}(B_2), \dots, \text{FOLLOW}(B_n)$ 两两不相交，则项目集 I 中的冲突可以按以下原则解决：

- 若 $a \in \{a_1, a_2, \dots, a_m\}$ ，则移进 a ；
- 若 $a \in \text{FOLLOW}(B_i)$ ，则用产生式 $B_i \rightarrow \gamma_i$ 归约；
- 其他情况报错。

如果有交集怎么办？

LR(1) 分析法

- SLR 分析法无法解决冲突的原因：
 - SLR 只是简单地考察下一个输入符号 b 是否属于与归约项目 $A \rightarrow \alpha$ 相关联的 $\text{FOLLOW}(A)$ ，但 $b \in \text{FOLLOW}(A)$ 只是归约 α 的一个必要条件，而非充分条件。
- 解决方法：通过分裂状态，使 LR 分析器的每个状态 $A \rightarrow \alpha \cdot$ 能确切知道句柄 α 后紧跟哪些终结符时才能把 α 归约成 A 。
- 对于产生式 $A \rightarrow \alpha$ 的归约，在不同的使用位置， A 会要求不同的后继符号。

规范 LR(1) 项

LR(1) 项是一个形如

$$[A \rightarrow \alpha \cdot \beta, a]$$

的二元组，其中

- $A \rightarrow \alpha \cdot \beta$ 与 LR(0) 项相同，表示产生式的分析进度；
- a 是向前看符号 (Lookahead Symbol)，为终结符或 \$。

a 也被称作搜索符，是归约条件：

- 在 β 非空时，不考虑 a ；
- 在 β 为空时（即当前为归约项目），则只有当前输入为 a 时，方可按 $A \rightarrow \alpha$ 进行归约，其中 a 的集合总是 FOLLOW(A) 的子集（可能是真子集）。

特别地，如果 $[A \rightarrow \alpha \cdot \beta, a] \in I$ 且 $[A \rightarrow \alpha \cdot \beta, b] \in I$ ，二者可合写为

$$[A \rightarrow \alpha \cdot \beta, a/b].$$

LR(1) 项集的闭包

Algorithm 12: LR(1) 项集闭包的计算

Input: 项集 I

Output: 项集闭包 $CLOSURE(I)$

```
1 while true do
2    $I' \leftarrow \emptyset$ ;
3   foreach  $[A \rightarrow \alpha \cdot B\beta, a] \in I$  do
4     foreach  $B$  的所有产生式  $B \rightarrow \gamma$  do
5       foreach  $b \in FIRST(\beta a)$  do
6         if  $\neg [B \rightarrow \cdot \gamma, b] \in I$  then
7            $I' \leftarrow I' \cup \{[B \rightarrow \cdot \gamma, b]\}$ ;
8   if  $I' = \emptyset$  then
9     return  $I \cup I'$ ;
10   $I \leftarrow I \cup I'$ ;
```

数的计算

```

1  $J \leftarrow \emptyset;$ 
2 foreach  $[A \rightarrow \alpha \cdot X\beta, a] \in I$  do
3    $J \leftarrow J \cup \{[A \rightarrow \alpha X \cdot \beta, a]\};$ 
4 return  $\text{CLOSURE}(J);$ 

```

LR(1) 项集族的构建

Algorithm 14: 构建 LR(1) 项集族

```
1  $\mathcal{C} \leftarrow \text{CLOSURE}(\{[S' \rightarrow S, \$]\});$   
2 repeat  
3   foreach  $I \in \mathcal{C}$  do  
4     foreach  $X \in V_T \cup V_N$  do  
5       if  $\text{GOTO}(I, X) \neq \emptyset \wedge \text{GOTO}(I, X) \notin \mathcal{C}$  then  
6          $\mathcal{C} \leftarrow \mathcal{C} \cup \text{GOTO}(I, X);$   
7 until 没有新的项集被加入  $\mathcal{C};$   
8 return  $\mathcal{C};$ 
```

LR(1) 自动机

LR(1) 自动机的形式化定义为

$$\mathcal{M} = (\mathcal{C}, V_T \cup V_N, \text{GOTO}, I_0, F),$$

其中

- $\mathcal{C} = \{I_0\} \cup \{J \mid \exists I \in \mathcal{C}, X \in V_T \cup V_N : J = \text{GOTO}(I, X)\}$
- $I_0 = \text{CLOSURE}(\{[S' \rightarrow \cdot S, \$]\})$
- $F = \{\text{CLOSURE}(\{[S' \rightarrow S, \$]\})\}$

LR(1) 分析表构建

- ① 构造 G' 的规范 LR(1) 项集族 $\mathcal{C} = \{I_0, I_1, \dots, I_n\}$;
- ② 根据 I_i 构造得到状态 i 。状态 i 的语法分析动作按照下面的方法决定：
 - $[A \rightarrow \alpha \cdot a\beta, b] \in I_i \wedge \text{GOTO}(I_i, a) = I_j \rightarrow \text{ACTION}[i, a] = s_j$;
 - $[A \rightarrow \alpha \cdot B\beta, b] \in I_i \wedge \text{GOTO}(I_i, B) = I_j \rightarrow \text{GOTO}[i, B] = j$;
 - $[A \rightarrow \alpha \cdot, a] \in I_i \wedge A \neq S' \rightarrow \text{ACTION}[i, a] = r_j$ (j 是产生式 $A \rightarrow \alpha$ 的编号);
 - $[S' \rightarrow S \cdot, \$] \in I_i \rightarrow \text{ACTION}[i, \$] = acc$;
- ③ 其他没有定义的所有条目都设置为 “error”。

如果 LR(1) 分析表中没有语法分析动作冲突，那么给定的文法就称为**LR(1) 文法**。

LALR(1) 分析器

- LALR 分析的核心思想：寻找同心集，并将它们合并成一个项集。
 - 可以压缩 LR(1) 分析表中的状态数，从而可能得到 LALR 分析表；
 - 压缩可能导致归约/归约冲突 (此时文法不是 LALR)；
 - 压缩不会导致移入/归约冲突。
- **同心项**：具有相同**核心** (LR(1) 项目集的第一分量集合) 的项集，即除展望符外，两个 LR(1) 项集是相同的。

LALR 表怎么建

- ① 暴力建：先构建规范 LR(1) 项集族，然后从最简单的单归约项开始逐步合并同心集，最终得到 LALR 的项集族并构建分析表；
- ② 基于 LR(0) 进行构建：
 - ① 使用 DeRemer-Pennello 算法，通过两次遍历（向前传播和反向传播）高效计算向前看符号；
 - ② 使用位向量（Bitmap）优化符号集合；
 - ③ 惰性求值（Lazy Evaluation）：仅在需要时生成特定状态的向前看符号，减少冗余计算。

LR(0)	SLR(1)	LR(1)	LALR(1)
✓	✓	✓	✓
✓	✓	✓	✓
	✓		
		✓	✓
✓	✓		
		✓	✓
			✓
✓	✓	✓	✓

1)	LR(1)	LALR(1)
W 集合	看搜索符/展望符	

LR(0)	SLR(1)	LR(1)	LALR(1)
填满行	看 FOLLOW 集合	看搜索符/展望符	